The picture can't be displayed.

Technical Documentation

RAG Customer Support Assistant

Prepared By
Sannidhya Milind Bahulekar

Internship ID
IN226066402

College / University
Ajeenkya D.Y. Patil University

Internship Organization
Innomatics Research Labs

Submission Date
24 April 2025

TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

System Ex

Contents

1. Introduction 3
1.1 What is Retrieval-Augmented Generation (RAG)? 3
1.2 Why is RAG Needed Here? 3
1.3 Use Case Overview 3
2. System Architecture Explanation 3
2.1 Overview of Component Interactions 3
2.2 Why a Graph-Based Workflow? 4
3. Design Decisions 4
3.1 Chunk Size Choice 4
3.2 Embedding Strategy 4
3.3 Retrieval Approach 5
3.4 Prompt Design Logic 5
4. Workflow Explanation 5
4.1 LangGraph Usage 5
4.2 Node Responsibilities 5
4.3 State Transitions 6
5. Conditional Logic & Intent Detection 6
5.1 Intent Detection 6
5.2 Confidence Score Derivation 6
6. HITL Implementation 6
6.1 Role of Human Intervention 6
6.2 Escalation Flow 7
6.3 Benefits and Limitations 7
7. Challenges & Trade-offs 7
8. Testing Strategy 7
8.1 Test Categories 8
9. Future Enhancements 8
9.1 Multi-Document Support 8
9.2 Feedback Loop 8
9.3 Conversation Memory 9
9.4 Production Deployment 9
10. Conclusion 9

1. Introduction

1.1 What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation is a design pattern for large language model (LLM) applications that addresses one of the most fundamental limitations of standalone LLMs: they only know what they were trained on. A model trained in 2023 knows nothing about a company's 2024 return policy. A model fine-tuned on general data knows nothing about the specific clauses in a particular insurance document.

RAG solves this by splitting the problem into two steps. First, a retrieval step uses semantic search to find the most relevant pieces of information from an external knowledge base. Then, a generation step feeds that retrieved information directly into the LLM prompt, grounding the model's response in actual evidence rather than memorized patterns.

> *The key insight of RAG: instead of training the model to know everything, we train it to use what it is given. This makes the system updatable without retraining, auditable by design, and far less prone to confident hallucination.*

1.2 Why is RAG Needed Here?

The customer support use case is a near-perfect fit for RAG for several reasons. Support knowledge bases change frequently — products are updated, policies are revised, pricing changes. Retraining an LLM every time a document changes is impractical and expensive. With RAG, the knowledge base is updated by simply re-ingesting the new PDF; the model itself never changes.

Additionally, customer support requires high accuracy and traceability. When a user asks about the return policy, the answer needs to come from the actual policy document, not from the model's statistical approximation of what return policies usually say. RAG enables source citation — every answer can point back to the specific chunk it was derived from.

This use case was chosen for my internship at Innomatics Research Labs because it combines retrieval, generation, and human oversight — covering all mandatory concepts required for this project.

1.3 Use Case Overview

This system is designed for a scenario where a company has a product manual, FAQ document, or policy PDF that forms the basis of its customer support knowledge. Users interact with the system through a chat interface, asking questions in natural language. The system retrieves the most relevant sections of the document and generates a clear, grounded answer. When the system lacks confidence — perhaps because the question falls outside the document's scope, or requires judgment beyond the document's content — it routes the query to a human support agent who resolves it directly.

2. System Architecture Explanation

2.1 Overview of Component Interactions

The system operates across two distinct timelines: an offline ingestion phase that runs once when the knowledge base is set up, and an online query phase that runs for every user interaction.

During ingestion, the PDF is loaded, chunked, embedded, and stored in ChromaDB. This is a one-time cost — or repeated when the knowledge base is updated. The output of ingestion is a populated vector store that can answer queries immediately.

During the query phase, the user's input flows through five stages: embedding the query, retrieving relevant chunks, generating a candidate answer, evaluating confidence, and either delivering the answer or escalating to a human. The LangGraph workflow engine coordinates all five stages, maintaining a state object that is passed between nodes.

2.2 Why a Graph-Based Workflow?

A naive implementation of this pipeline might be a simple Python function that calls each step in sequence. This works for the happy path, but breaks down when you introduce branching logic. What happens when the retrieval returns nothing? What if the LLM response is low-confidence? What if the user follows up on a previous question?

LangGraph addresses this by representing the workflow as a directed graph with typed state. Each node modifies the state object and passes it forward. Conditional edges evaluate the current state and decide which node to execute next. This makes the control flow explicit, visible, and testable. It also enables the HITL suspend-and-resume pattern, which would be very difficult to implement cleanly in a linear function call chain.

3. Design Decisions

3.1 Chunk Size Choice

The choice of 512 tokens with 50-token overlap was reached through reasoning about the nature of the source documents and the embedding model's behavior.

Most factual customer support questions can be answered from a single coherent paragraph — a policy clause, a product specification, an FAQ answer. These are typically 100 to 300 words. A 512-token chunk is large enough to contain one complete thought with surrounding context, while being small enough that the embedding vector meaningfully represents the specific content rather than averaging across unrelated topics.

The 50-token overlap ensures that information at chunk boundaries is captured. Without overlap, a sentence that spans the boundary between chunk N and chunk N+1 would never be retrieved as a complete unit.

> *A practical heuristic: if your document contains mostly short factual paragraphs, use smaller chunks (256-512 tokens). If it contains narrative explanations that span multiple paragraphs, use larger chunks (512-1024 tokens) with proportionally larger overlap.*

3.2 Embedding Strategy

The all-MiniLM-L6-v2 model was chosen as the default embedding model for its balance of speed, cost, and quality. It runs entirely on-device with no API calls, produces 384-dimensional vectors, and is well-suited to short factual passages. Its primary limitation is that it was not fine-tuned on domain-specific customer support text, which means retrieval accuracy for highly technical or jargon-heavy documents may be lower than expected.

OpenAI's text-embedding-3-small is offered as an alternative for cases where retrieval accuracy is paramount. The trade-off is cost (API calls per query) and latency (network round-trip). Both models share the same interface in this system, making them interchangeable via a configuration flag.

3.3 Retrieval Approach

The system uses dense retrieval via cosine similarity — the standard approach for RAG systems. The query embedding and chunk embeddings exist in the same vector space, so semantically similar texts will have high cosine similarity regardless of exact word overlap. This handles paraphrasing naturally: a user asking 'how long do I have to return something' will retrieve chunks containing 'return window is 30 days' even though none of the exact words match.

A BM25 sparse retrieval layer (keyword-based) was considered but not included in the initial design. Hybrid retrieval (combining dense and sparse scores) would improve recall for queries that rely on specific product codes or model numbers, and is flagged as a future enhancement.

3.4 Prompt Design Logic

The system prompt follows three design principles. First, grounding: the model is explicitly told to answer only from the provided context. This is the most important constraint — without it, the model will fill gaps with plausible-sounding information from its training data. Second, fallback signaling: the model is told to say 'I am not able to answer this from the available documentation' when the context is insufficient. This signal is detected programmatically to trigger escalation. Third, conciseness: the model is not asked to explain its reasoning in full — customers want direct answers, not verbose explanations.

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is initialized with a StateGraph object parameterized by the GraphState TypedDict. Nodes are registered as functions that accept a GraphState and return a partial update to it. Edges are added explicitly between nodes, with conditional edges taking a function that returns the name of the next node based on the current state.

The graph is compiled with graph.compile() and executed with graph.invoke(initial_state). For the HITL path, graph.stream() or interrupt() is used to suspend execution mid-graph and resume it with an updated state when the human response arrives.

4.2 Node Responsibilities

Node	Detailed Responsibility
input_node	Validates that the query is non-empty and meets minimum length. Calls the Embedding Module to convert the query string to a vector. Stores the result in state.embedded_query. Generates a unique session_id and records a timestamp.
process_node	Calls the Retrieval Module with state.embedded_query to get top-k chunks. Builds the LLM prompt from those chunks and state.query. Calls the LLM API. Parses the response for the answer text and a confidence signal. Stores all results in state.
output_node	Takes state.answer (or state.human_response if escalated). Formats the response with source

	citations. Prints or returns to the user. Logs the complete interaction.
hitl_node	Generates a ticket ID. Writes the ticket to storage. Calls interrupt() to suspend the graph. Returns ticket ID to user. On resume, reads state.human_response and passes to output_node.

4.3 State Transitions

The GraphState object flows through the graph as an immutable record — each node returns only the keys it modifies, and LangGraph merges them into the current state. This design ensures that earlier node outputs are always available to later nodes. For example, the output_node can access state.retrieved_chunks (set by process_node) to generate source citations even though it does not call the retriever itself.

5. Conditional Logic & Intent Detection

5.1 Intent Detection

Intent detection in this system is intentionally lightweight. Rather than training a separate intent classification model, the system uses two signals to detect escalation-worthy queries: the confidence score from retrieval, and a keyword filter applied to the query string.

The keyword filter checks for terms associated with high-stakes or emotionally sensitive requests — words like 'cancel', 'refund', 'complaint', 'legal', 'fraud', 'urgent', and 'manager'. When these terms appear, the system escalates regardless of the confidence score. This is a conservative design choice: false positives (escalating a query that the system could have handled) are preferable to false negatives (auto-answering a query about a billing dispute incorrectly).

5.2 Confidence Score Derivation

The confidence score is derived from the similarity scores returned by ChromaDB. A raw L2 distance is converted to a normalized similarity score between 0 and 1 using the formula: score = 1 / (1 + distance). The minimum score across the top-k retrieved chunks is used as the overall confidence, rather than the average. This is a conservative choice — if any of the retrieved chunks is a poor match, the system treats the overall retrieval as low-confidence. The threshold of 0.75 was chosen based on empirical testing with sample queries; it should be recalibrated on real data before production deployment.

6. HITL Implementation

6.1 Role of Human Intervention

Human intervention in this system serves two purposes. The immediate purpose is to handle queries that the automated system cannot answer reliably. The longer-term purpose is to generate labeled training data: every human-resolved ticket is a ground-truth example of a difficult query and

its correct answer, which can be used to improve the system's retrieval accuracy, refine the confidence threshold, or fine-tune the LLM in future iterations.

6.2 Escalation Flow

When the routing condition is met, the hitl_node creates a ticket and calls LangGraph's interrupt() mechanism. This suspends the graph's execution thread. The user receives an immediate response: 'Your query has been escalated to our support team. Your reference number is TKT-20250424-0017. We will follow up within 24 hours.'

The support agent accesses an agent dashboard (in the prototype, a CLI command) that displays the open tickets with full context — the original query, the retrieved chunks, and the LLM's attempted answer. The agent reviews this context and enters a response. The system resumes the graph with the human response injected into state.human_response, and the output_node delivers it to the user.

6.3 Benefits and Limitations

The primary benefit of the HITL design is reliability: the system never returns a wrong answer with high confidence, because low-confidence paths are redirected to humans. This is especially important in customer support contexts where a wrong answer about a refund or warranty could create real business liability.

The primary limitation is latency. The HITL path introduces a dependency on human response time, which breaks the expectation of an instant automated response. This is mitigated in the current design by providing an immediate acknowledgement with a ticket reference, setting the expectation of a follow-up within a defined SLA window. In higher-volume deployments, a dedicated support queue with SLA tracking would be needed.

7. Challenges & Trade-offs

Challenge	Trade-off	Resolution Approach
Retrieval accuracy vs speed	Larger k=10 increases recall but slows prompt generation	Use k=4 by default; tune per domain
Chunk size vs context quality	Smaller chunks are precise but lose surrounding context	Use 512 tokens with 50-token overlap
Confidence threshold calibration	Too high = over-escalation; too low = wrong auto-answers	Set 0.75 as default; recalibrate on labeled test set
Cost vs performance	OpenAI embeddings are more accurate but paid	Use MiniLM by default; switch to OpenAI for prod
HITL latency	Human response can take hours, breaking UX expectations	Immediate acknowledgement + SLA-bound follow-up
Hallucination risk	LLM may ignore grounding constraint	Explicit system prompt + fallback phrase detection

8. Testing Strategy

8.1 Test Categories

Testing is organized into four categories that together validate the system's behavior across the full range of expected inputs.

Category 1: High-Confidence Factual Queries

These are queries where the answer exists verbatim or near-verbatim in the knowledge base. The system should return the correct answer with high confidence and source citation.

- 'What is the warranty period for the product?'
- 'What is your return policy?'
- 'What payment methods do you accept?'

Category 2: Paraphrased Queries

These queries ask for the same information as Category 1 but with different wording. Tests whether the semantic embedding retrieves the right chunks despite vocabulary mismatch.

- 'How long can I return something?' (vs 'What is your return policy?')
- 'Does the warranty cover accidental damage?'

Category 3: Low-Confidence / Out-of-Scope Queries

These queries cannot be answered from the knowledge base. The system should either escalate or clearly state that the information is not available, rather than hallucinating an answer.

- 'What is the CEO's phone number?'
- 'Do you ship to Antarctica?'

Category 4: Escalation-Keyword Queries

These queries contain high-stakes keywords and should always trigger HITL escalation regardless of confidence score.

- 'I want to cancel my account and get a refund for all charges.'
- 'I am filing a complaint — this is a legal matter.'

9. Future Enhancements

9.1 Multi-Document Support

The current system is designed around a single PDF knowledge base. Extending it to support multiple documents requires adding a document registry that tracks which document each chunk came from, and updating the metadata stored in ChromaDB accordingly. The retriever would then return source-level attribution alongside each chunk. The ingestion pipeline would need to handle incremental updates — adding or removing individual documents without reprocessing the entire collection.

9.2 Feedback Loop

Every human-resolved ticket in the HITL system is a labeled training example: a difficult query paired with a correct answer. Over time, this dataset can be used to fine-tune the embedding model or the LLM on domain-specific queries, improving both retrieval accuracy and answer quality. A feedback loop mechanism could also track user satisfaction signals (thumbs up/down ratings) to identify systematic failure modes in the automated path.

9.3 Conversation Memory

The current system treats every query as stateless — it has no memory of what was discussed earlier in the conversation. Adding conversation memory would allow the system to handle follow-up questions like 'Can you tell me more about that?' or 'What about for international orders?' This requires storing the conversation history in the graph state and including it in the retrieval and generation steps.

9.4 Production Deployment

A production deployment of this system would replace the CLI interface with a FastAPI backend and a React or Next.js frontend. ChromaDB would be migrated to server mode or replaced with Weaviate for high-concurrency support. The LangGraph workflow would run as a persistent service with a job queue for HITL ticket management. Monitoring and observability would be added via LangSmith or a similar tracing platform to track retrieval quality, latency, and escalation rates over time.

10. Conclusion

This system demonstrates that RAG is not just a chatbot pattern — it is a principled engineering approach to building AI systems that are grounded, auditable, and maintainable. By separating retrieval from generation, we gain control over what the model knows. By adding a graph-based workflow, we make the system's decision-making explicit and testable. By integrating HITL escalation, we ensure that the system's failures are handled gracefully rather than silently.

The design prioritizes correctness over coverage: it is better to escalate a query to a human than to answer it wrongly. This philosophy is appropriate for a customer support context where incorrect information erodes trust and creates real-world consequences. As the system matures and its failure modes are better understood, the confidence threshold can be raised and the escalation rate reduced — moving gradually from human-assisted to fully automated, with data to support every step of that transition.

This internship project gave me hands-on experience designing a production-grade AI system from scratch — from PDF ingestion to LangGraph workflow orchestration and HITL escalation.